

THE MICROPROCESSORS & MICROCONTROLLERS

Instructor: The Tung Than

Student's name: Do Quoc Huy

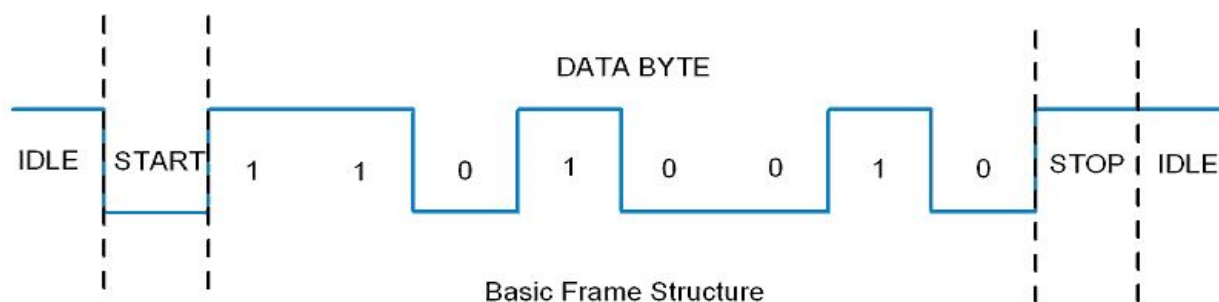
Student's code: 23520604

PRACTICE EXERCISE #4:

USING UART

I. Student preparation

- Knowledge of how to install and use UART.
- Giao tiếp UART là chuẩn giao tiếp nối tiếp được sử dụng phổ biến trong các dòng VĐK. Mặc dù có tốc độ truyền không cao, nhưng ngược lại giao tiếp UART có phần cứng gọn nhẹ, dễ thiết kế và giá thành thấp. Giao tiếp UART được sử dụng nhiều trong các ứng dụng như giao tiếp máy tính, giao tiếp module SIM, module GPS...
- 1. Các phương thức truyền thông nối tiếp
 - **Truyền đồng bộ:** Truyền dữ liệu theo khung cấu trúc đã được định nghĩa trước.
 - **Truyền không đồng bộ:** Truyền từng byte dữ liệu một cách độc lập trong khung cấu trúc.
 - UART thuộc dạng truyền không đồng bộ. Giao thức này sử dụng ba chân kết nối chính:
 - Chân RxD (Serial data receive pin).
 - Chân TxD (Serial data transmit pin).
 - Chân GND (Ground pin, nối chung).
 - Để kết nối giữa hai vi điều khiển (VĐK) sử dụng giao thức UART, chúng ta chỉ cần nối chung chân GND và kết nối chân TxD của VĐK này với RxD của VĐK còn lại và ngược lại.
- 2. Cấu trúc khung truyền UART
 - START bit: Khi đường truyền ở trạng thái cao (=1), bit START sẽ kéo xuống thấp (=0) để bắt đầu truyền dữ liệu.
 - Gói các bit dữ liệu: Số bit trong mỗi gói có thể từ 5 đến 9, nhưng thông thường là 8 bit.
 - Parity bit: Dùng để kiểm tra tính chính xác của dữ liệu, có thể là Even Parity (chẵn) hoặc Odd Parity (lẻ).
 - STOP bit: Một hoặc hai bit STOP sẽ luôn có giá trị bằng 1, báo hiệu kết thúc truyền.



3. Các thanh ghi cấu hình giao tiếp UART trong VĐK 8051.

a. Thanh ghi cấu hình cổng nối tiếp SCON.

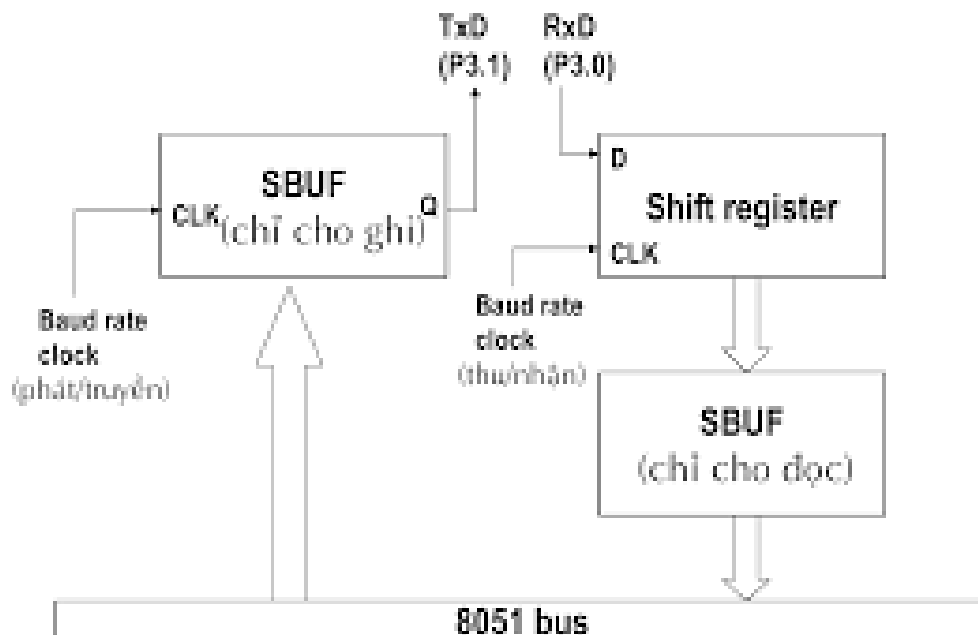
- Đây là thanh ghi 8 bit, chứa các bit sử dụng để cài đặt chế độ của giao tiếp UART và các bit cờ báo trạng thái truyền và nhận.

SM0	SM1	SM2	REN	TB8	RB8	TI	RI
-----	-----	-----	-----	-----	-----	----	----

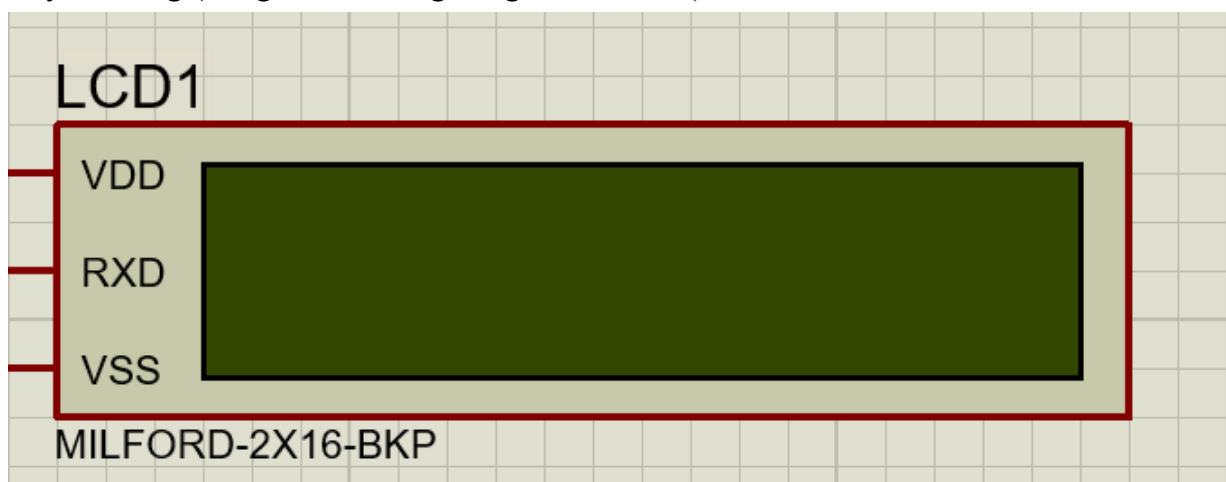
- SM0, SM1, SM2: Đây là 2 bit sử dụng để xác định các chế độ làm việc. Có 4 chế độ truyền UART trong 8051, được miêu tả trong bảng sau.

SM0	SM1	Chế độ	Khung truyền	Tốc độ Baudrate
0	0	0	8-bit Shift Register	Oscillator / 12
0	1	1	8-bit UART	Xác định qua TIMER1
1	0	2	9-bit UART	Oscillator/ 32
1	1	3	9-bit UART	Xác định qua TIMER1

- REN: Bit cho phép nhận dữ liệu, khi sử dụng chức năng nhận dữ liệu qua cổng UART, phải bật bit này lên 1.
 - TB8, RB8: Bit thứ 9 của truyền/nhận trong chế độ truyền 9 bit.
 - TI, RI: Cờ báo ngắt truyền, ngắt nhận.
- #### b. Thanh ghi truyền/nhận dữ liệu: SBUF.
- SBUF là thanh ghi 8 bit, được sử dụng chung cho cả truyền và nhận dữ liệu trong UART.
 - Để truyền dữ liệu, chúng ta chỉ cần ghi dữ liệu vào thanh ghi SBUF.
 - Để nhận dữ liệu chúng ta đọc giá trị từ thanh ghi SBUF.



- Knowledge of how to use Serial UART LCD
- **Serial UART LCD** là loại màn hình LCD (thường là LCD 16x2 hoặc 20x4) được tích hợp thêm giao tiếp nối tiếp **UART** (Universal Asynchronous Receiver/Transmitter) để truyền dữ liệu thay vì dùng nhiều chân như loại LCD truyền thống (dùng chế độ song song 4 hoặc 8 bit).



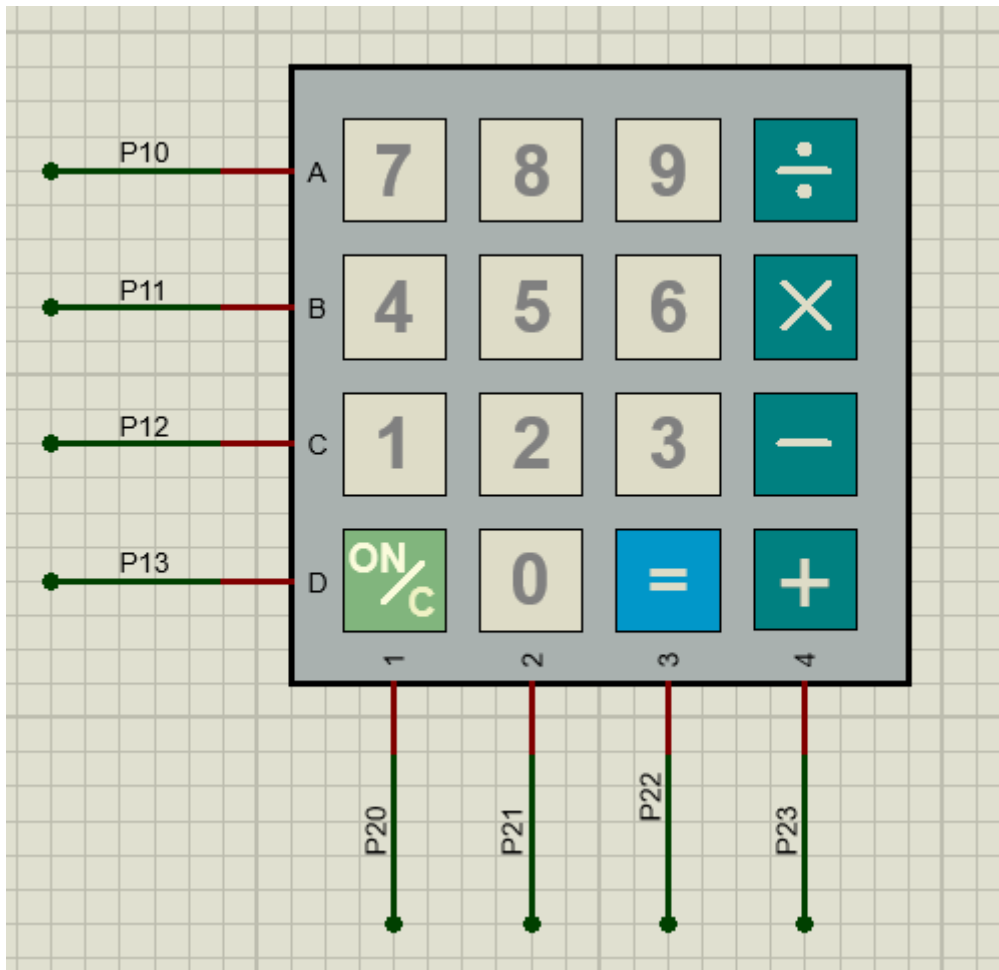
- Cổng kết nối:

LCD UART	Vi điều khiển
VDD	5V
VSS	GND
RX	TX của MCU

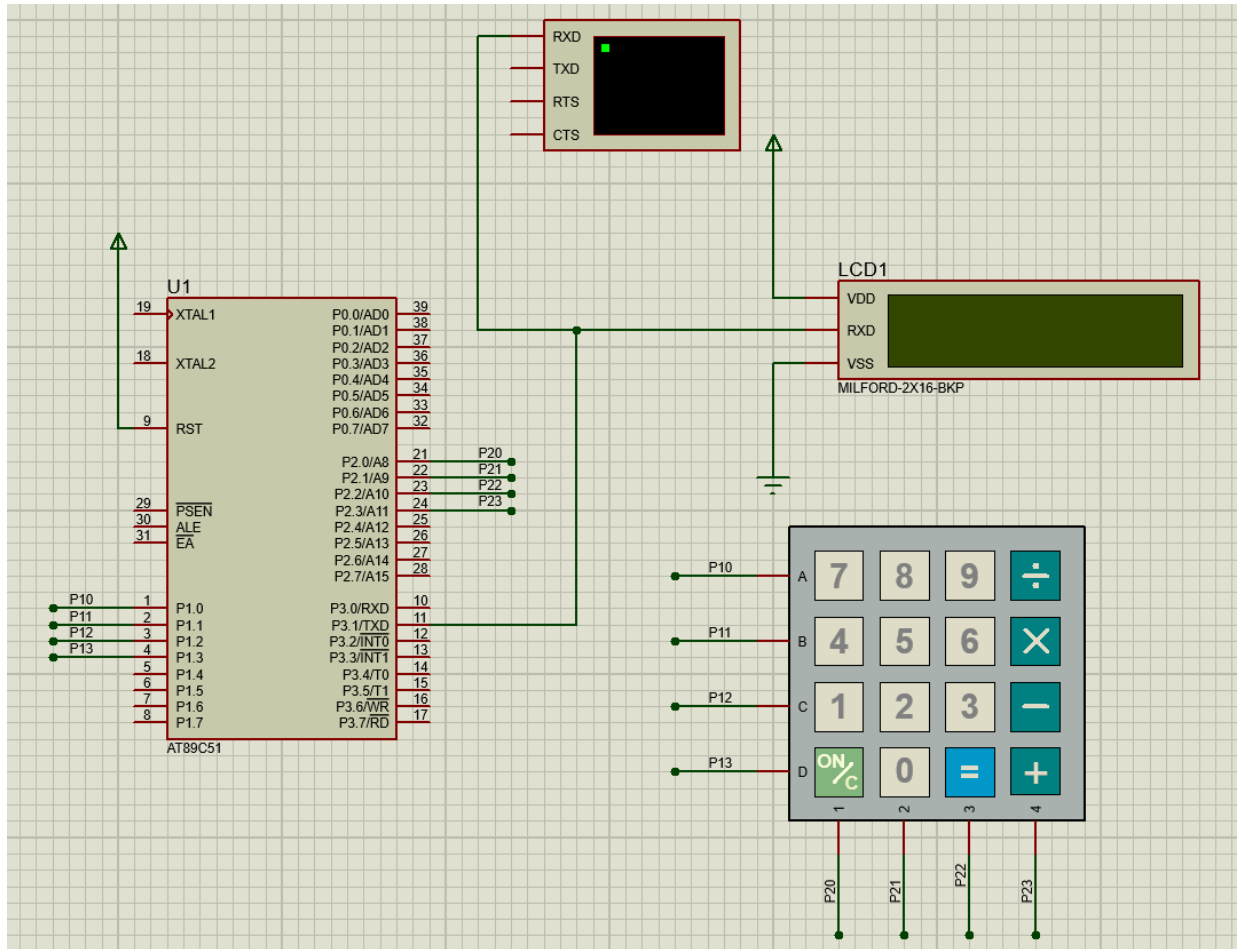
II. Practice content

1. Design a 4x4 keyboard set including the following buttons:

- From 0 to 9
- The signs + - * /
- Sign =
- Reset button



- Using AT89C51/AT89C52 in combination with the module just designed above to design a handheld calculator, display calculations and results on an LCD that receives data via UART.



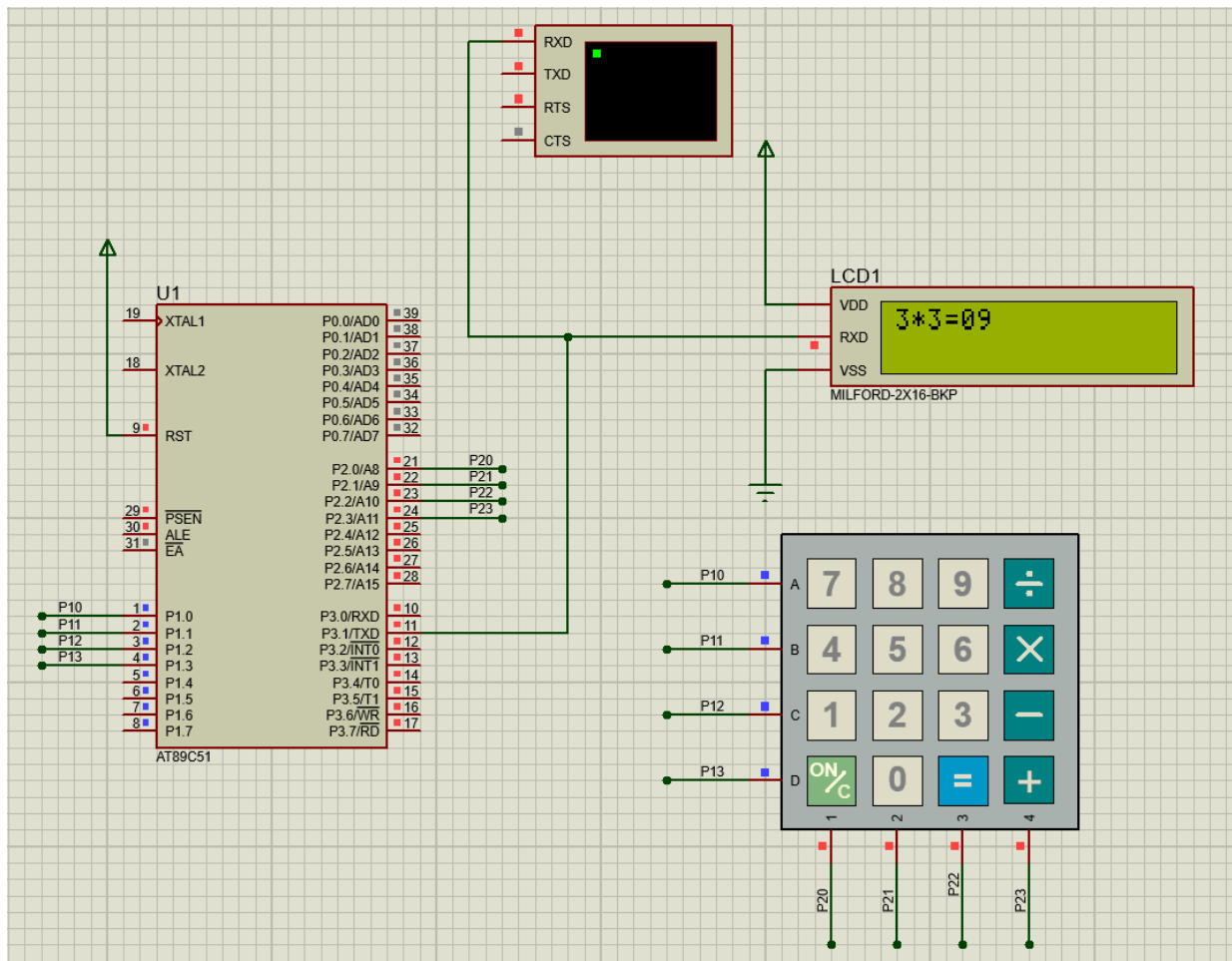
III. Report

Compress design files and report files into a file named as follows:

[<LAB...>]-[<Student code>]-Full name

The required report file contains the following contents:

1. Design result (screenshot and pasted in the report).



2. Explain the operating principle of the effects, accompanied by a video (send a Google Drive link) to demonstrate the circuit operation in case the instructor cannot run the design file.

- Link drive:

<https://drive.google.com/drive/folders/1yPwSQZeIUtRiw2JIjmNTOCfDr1tx7EyO?usp=sharing>

- Giải thích:

- Hàm khởi tạo

```
$NOMOD51
$INCLUDE (8051.MCU)

ORG 00H
ORG 30H
TEMP EQU R0
OP_A EQU R1
OP EQU R2
OP_B EQU R3
```

- Khởi tạo các thanh ghi để lưu các toán hạng đếm số lần nhập

- Hàm main:

- Đặt TEMP = 0 dùng để đếm số phím

<pre> START: MOV TEMP, #0 MAIN: MOV P1, #0 ;DUA P1 MOV A, P2 ;DOC GIA ANL A, #0FH ; AND P CJNE A, #0FH, MAIN </pre>	nhập. - P1 là cổng điều khiển các hàng (row) keypad, ban đầu để tất cả về 0. - Đọc trạng thái cổng P2 (cổng các cột), AND với 0x0F để chỉ lấy 4 chân thấp (4 cột). - Nếu các cột đều là 1 (0x0F), tức không có phím nào được nhấn, tiếp tục lặp lại MAIN. - Nếu có phím nhấn (cột có mức thấp), thoát vòng MAIN để xử lý tiếp.
- Mảng bàn phím <pre> ORG 300H ;LUU MANG BANG PHIM MAHANG0: DB '7','8','9','/' MAHANG1: DB '4','5','6','*' MAHANG2: DB '1','2','3','-' MAHANG3: DB 'C','0','=','+' END </pre>	- Mã hàng được theo thứ tự kết nối các chân ở P2
- Hàm chống dôi phím <pre> CHONGDOIPHIM: LCALL DELAY MOV A, P2 ANL A, #0FH CJNE A, #0FH, DUOCNHAN SJMP CHONGDOIPHIM DUOCNHAN: LCALL DELAY MOV A, P2 ANL A, #0FH CJNE A, #0FH, CHECKCOL SJMP CHONGDOIPHIM </pre>	- Gọi hàm DELAY để chống rung phím theo giải thuật chống dôi - Đọc lại cổng P2, nếu vẫn có phím nhấn, tiếp tục xử lý DUOCNHAN - Nếu không có phím được nhấn thì chờ tín hiệu (quay lại hàm chống dôi) Hàm DUOCNHAN: - Đọc lại P2, nếu vẫn có phím nhấn thì nhảy sang kiểm tra cột CHECKCOL - Nếu không còn phím nhấn nữa thì quay lại CHONGDOI để chờ tín hiệu.

- Hàm kiểm tra cột <pre> CHECKCOL: MOV P1,#0FEH ;11111110 MOV A,P2 ANL A,#0FH CJNE A,#0FH,SCAN_R0 MOV P1,#0FDH ;11111101 MOV A,P2 ANL A,#0FH ;00001101 CJNE A,#0FH,SCAN_R1 MOV P1,#0FBH ;11111011 MOV A,P2 ANL A,#0FH CJNE A,#0FH,SCAN_R2 MOV P1,#0F7H ;11110111 MOV A,P2 ANL A,#0FH CJNE A,#0FH,SCAN_R3 LJMP CHONGDOIPHIM </pre>	- Thiết lập bằng cách so sánh giả thiết (P1) và thực tế (P2). Nếu 2 cổng có giá trị giống nhau thì ta nhận ra là đã có phím được nhấn. - Nếu phát hiện có phím nhấn, nhảy đến hàm tương ứng SCAN_Rx để xác định phím nhấn ở dòng đó. - Nếu không có bất kì phím nào được nhấn, ta chờ tín hiệu.
- Hàm quét hàng <pre> SCAN_R0: MOV DPTR,#MAHANG0 LCALL SCAN LJMP MAIN SCAN_R1: MOV DPTR,#MAHANG1 LCALL SCAN LJMP MAIN SCAN_R2: MOV DPTR,#MAHANG2 LCALL SCAN LJMP MAIN SCAN_R3: MOV DPTR,#MAHANG3 LCALL SCAN LJMP MAIN </pre>	- Gán địa chỉ bảng mã ký tự tương ứng với hàng đó cho DPTR - Gọi hàm SCAN để xác định cột phím
- Hàm xác định vị trí chính xác nút	- Dịch phải xoay thanh ghi A (chứa

bấm <pre> SCAN: RRC A JNC MATCH INC DPTR SJMP SCAN MATCH: INC TEMP MOV A, #0 MOVC A, @A+DPTR ACALL DISPLAY LCALL CLEAR LCALL TINHTOAN RET </pre>	trạng thái cột). - Nếu bit carry = 0 nghĩa là phát hiện cột nhân, nhảy MATCH. - Nếu không thấy, tăng DPTR để tới ký tự tiếp theo trong bảng mã - Khi tìm được, tăng TEMP - Lấy ký tự từ bảng mã, gọi hàm DISPLAY hiển thị. - Gọi hàm TINHTOAN để xử lý phép tính.
- Hàm Clear <pre> CLEAR: CJNE A, #'C', EXIT2 MOV TEMP, #0 MOV A, #254 LCALL DISPLAY MOV A, #1 LCALL DISPLAY </pre>	- Nếu ký tự vừa nhập là 'C' thì xóa biến TEMP (reset bộ đếm). - Nếu không phải 'C', thoát ngay.
- Hàm hiển thị qua UART <pre> DISPLAY: MOV TMOD, #20H MOV TH1, #0FDH MOV SCON, #50H SETB TR1 MOV SBUF, A JNB TI, \$ CLR TI RET </pre>	- Cấu hình Timer1 để tạo baud rate cho UART - Gửi ký tự trong A ra cổng UART. - Đợi đến khi truyền xong mới trả về.
- Hàm tính toán các phép tính	- Hàm phức tạp xử lý các phép cộng, trừ, nhân, chia. - Phân tích biểu thức nhập vào theo các ký tự '+', '-', '*', '/', '='. - Thực hiện phép toán tương ứng giữa OP_A và OP_B.

<pre> TINHTOAN: BANG: CJNE A, # "=", NEXT CONG: CJNE OP, # "+", TRU MOV A, OP_A ADD A, OP_B LJMP RESULT TRU: CJNE OP, # "-", NHAN CLR C MOV A, OP_A SUBB A, OP_B LJMP RESULT NHAN: CJNE OP, # "*", CHIA MOV A, OP_A MOV B, OP_B MUL AB LJMP RESULT CHIA: CJNE OP, # "/", EXIT MOV A, OP_B CJNE A, #0, DIVIDE MOV A, #'E' ACALL DISPLAY MOV TEMP, #0 LJMP EXIT </pre>	Giải thích kỹ về phép chia: - Kiểm tra số chia, nếu bằng 0 thì in ra kí tự "E", khác 0 thì thực hiện chia - Ta nhân phần dư với 10 để lấy phần thập phân hiển thị. - Phần xử lí phép chia nhiều công đoạn do có nhiều trường hợp có thể xảy ra
--	---

<pre> DIVIDE: ;chia lay so thap MOV A, OP_A MOV B, OP_B DIV AB ADD A, #'0' ACALL DISPLAY ; phan du MOV R7, A ; MOV A, B ; MOV B, #10 MUL AB ; MOV B, OP_B ; DIV AB ; MOV R6, A MOV A, #'.' ACALL DISPLAY MOV A, R6 ADD A, #'0' ACALL DISPLAY LJMP EXIT </pre>	
- Hàm delay <pre> DELAY: LOOP2: MOV 8AH, #LOW(55536) MOV 8CH, #HIGH(55536) MOV 89H, #01H ;timer 1 mode 0 ; timer 0 mode 1 SETB 8CH LOOP3: JNB 8DH, LOOP3 CLR 8DH CLR 8CH RET </pre>	- T0 = 0xD8A0 - SETB 8CH ⇔ TR0 = 1: Bắt đầu chạy Timer0 - Hàm delay này chạy khoảng $10000 \times 11.0592\text{MHz} / 12 \sim 10\text{ms}$ -

- Hàm nhập theo thứ tự : OP_A OP_B <pre> ; Nhập theo thứ tự NEXT: CJNE TEMP, #1, NEXT1 SUBB A, #'0' MOV OP_A, A LJMP EXIT NEXT1: CJNE TEMP, #2, NEXT2 MOV OP, A LJMP EXIT NEXT2: CJNE TEMP, #3, EXIT SUBB A, #'0' MOV OP_B, A LJMP EXIT </pre>	
- Hàm xử lý kết quả âm dương	- Trường hợp dương: Chỉ cần chia và hiển thị phần thương và phần dư. - Trường hợp âm: Đảo bù 2, sau đó hiển thị dấu trừ trước phần thương và phần dư.

```
RESULT:
    JNB ACC.7, DUONG
    SJMP AM
```

```
DUONG:
    MOV B, #10
    DIV AB
    ADD A, #'0'
    LCALL DISPLAY
    MOV A, B
    ADD A, #'0'
    LCALL DISPLAY
    MOV TEMP, #0
    LJMP EXIT
```

```
AM:
    CPL A
    INC A
    MOV R5, A
    MOV A, #'-'
    LCALL DISPLAY
    MOV A, R5
    MOV B, #10
    DIV AB
    ADD A, #'0'
    LCALL DISPLAY
    MOV A, B
    ADD A, #'0'
    LCALL DISPLAY
    MOV TEMP, #0
    LJMP EXIT
```

```
EXIT:
    RET
```

3. Exercise report.